
WORKBENCH IT · POC INTERNO

SecondBrain

Pipeline técnico — Documento de referencia
para evaluación con el equipo

Versión: snapshot al 2026-05-16

Datos: medidos sobre la instalación real, no estimados

Hardware: i7-10th gen · 32 GB RAM · RTX 3070 Ti 8 GB VRAM · Windows
11 + WSL 2 + Docker Desktop

SecondBrain — Pipeline técnico (POC)

Estado al 2026-05-16 · Documento de referencia para evaluación del equipo Todos los números son medidos sobre la instalación real corriendo en el equipo de Damian (i7-10th gen, 32 GB RAM, RTX 3070 Ti, 8 GB VRAM, Windows 11 + WSL 2 + Docker Desktop).

0. Qué es este documento y qué NO es

Es una foto técnica del estado actual del POC: arquitectura, decisiones tomadas, cómo se encadenan los componentes, y métricas reales tomadas hoy. Sirve para:

- Que alguien nuevo entienda el sistema sin abrir el código
- Discutir trade-offs con el equipo antes de invertir en sprints más grandes
- Tener una base para escribir un ADR (Architecture Decision Record) cuando algo cambie

No es un manual de usuario ni un tutorial. Para eso está el `README.md`. Tampoco es un plan de roadmap — eso está en `docs/sprints.md`.

1. Resumen ejecutivo

SecondBrain es un sistema de **memoria aumentada privada**: indexa y procesa conversaciones de WhatsApp (y eventualmente otras fuentes) para responder preguntas en lenguaje natural sobre actividad personal y profesional. Todo el procesamiento es **100% local** — LLMs, embeddings, transcripción, OCR, vector DB; nada sale del equipo.

El sistema tiene cuatro flujos principales:

- 1. **Ingesta** → WhatsApp en vivo (bridge whatsapp-web.js) + exports históricos (.txt) + uploads manuales (audios, docs, imágenes)
- 2. **Procesamiento** → cola de 5 etapas que enriquecen los items: transcribe → extract → caption → embed → tagger
- 3. **Almacenamiento** → Postgres (estructurado), Qdrant (vectores), MinIO (archivos crudos)
- 4. **Retrieval (chat Q&A)** → query expansion → búsqueda híbrida (mensajes + facts) → generación con citas

Estado actual medido:

Métrica	Valor
Items totales en core.items	75.702
Items embebidos en Qdrant	75.690 (99,98%)
Items taggeados (procesamiento completo)	617 + 631 en cola
Personas canónicas	993
Empresas	79
Conversaciones	113
Facts estructurados	871
Promesas detectadas	28
Transacciones detectadas	11
Vectores en Qdrant messages	75.683
Vectores en Qdrant facts	780
Tiempo de respuesta del chat (warm)	~2-5 s

2. Hardware y la restricción que define todo el diseño

El equipo es un i7 10ma gen + 32 GB RAM + RTX 3070 Ti con 8 GB VRAM. Los 8 GB de VRAM son la restricción más dura del diseño. Modelos típicos que considerar:

- LLM de 4 B Q4 $\approx$ 3 - 4 GB en VRAM
- LLM de 8 B Q4 $\approx$ 5 - 6 GB en VRAM
- Whisper Large V3 Turbo $\approx$ 1.5 - 2 GB
- Embedding model 4 B Q4 $\approx$ 2.5 - 3 GB

Dos modelos de 8 B juntos NO entran. Tres cosas en GPU al mismo tiempo, menos.

Baseline real medido: - Sistema Windows + apps sin nada nuestro $\rightarrow \sim 0.7$ GB VRAM - Sistema + Whisper (cuando estaba en GPU) $\rightarrow 5.0$ GB (Whisper se comía 4.3 GB sin transcribir nada) - Sistema + Whisper en CPU + qwen 3 : 8 b cargado en Ollama $\rightarrow 6.0$ GB

Esto fue clave para optimizar el chat — ver § 1.0.

3. Stack de servicios

Todo corre en Docker Compose. Definido en `docker-compose.yml`.

Servicio	Imagen	Propósito	Recursos
<code>postgres</code>	<code>pgvector/pgvector:pg16</code>	DB relacional con 5 schemas (<code>core</code> , <code>media</code> , <code>processing</code> , <code>analytics</code> , <code>audit</code>)	CPU + RAM
<code>qdrant</code>	<code>qdrant/qdrant:latest</code>	Vector DB. Collections: <code>messages</code> , <code>facts</code>	CPU + RAM
<code>minio</code>	<code>minio/minio:latest</code>	Object storage (Vault de archivos crudos). Buckets: <code>raw</code> , <code>derived</code> , <code>exports</code>	CPU + disco
<code>ollama</code>	<code>ollama/ollama:latest</code>	Servidor LLM local. Acceso GPU	GPU
<code>whisper</code>	<code>onerahmet/openai-whisper-asr-websocket:latest</code>	Transcripción. CPU-only (decisión consciente, ver § 1 0)	CPU
<code>backend</code>	Python 3 . 1 2 + FastAPI + SQLAlchemy 2 + uv	API + worker continuo	CPU + RAM
<code>frontend</code>	Python 3 . 1 2 + Streamlit	Panel web (validación, debugging, UIs sencillas)	CPU
<code>bridge</code>	Node.js + whatsapp-web.js	Captura WhatsApp en vivo, descarga media	CPU

Aparte de Docker: - `panel/` — App de escritorio nativa (PySide 6) para orquestación, monitoreo y trigger manual. Reemplaza el browser para tareas de control.

4. Modelos LLM locales

Servicios por Ollama (`/api/generate` , `/api/embed` , `/api/chat`). Modelos descargados en disco:

Modelo	Tamaño	Familia	Params	Uso actual
<code>qwen3-embedding:4b</code>	2.50 GB	qwen 3	4.0 B	Embeddings (mensajes + facts). 2560 dim
<code>qwen3:4b</code>	2.50 GB	qwen 3	4.0 B	Reserva
<code>gemma3:4b</code>	3.34 GB	gemma 3	4.3 B	Reserva (multimodal, candidato a reemplazo de qwen 3 -vl)
<code>aya-expanse:8b</code>	5.06 GB	command-r	8.0 B	Reserva
<code>qwen3:8b</code>	5.23 GB	qwen 3	8.2 B	Chat principal + tagger . Ganador del benchmark con datos reales
<code>qwen3-vl:8b</code>	6.14 GB	qwen 3 vl	8.8 B	Visión (OCR + caption + entidades en imágenes). Solo se carga en ventana nocturna
<code>gemma4:e2b</code>	7.16 GB	gemma 4	5.1 B	Reserva
<code>gemma3:12b</code>	8.15 GB	gemma 3	12.2 B	Reserva
<code>gemma4:e4b</code>	9.61 GB	gemma 4	8.0 B	Reserva

Modelo de transcripción: Whisper Large V3 Turbo vía `faster_whisper` , ejecutado en CPU (decisión, ver § 1.0).

`OLLAMA_KEEP_ALIVE=30m` — los modelos se mantienen calientes 30 min después de usarse. Crucial para el chat: cuando vos preguntás y qwen 3:8b ya está caliente, el primer token sale en ~200 ms.

5. Pipeline de ingesta

Tres vías de entrada, todas terminan en la tabla `core.items` (un row = un mensaje/email/nota/audio/etc.):

5.1 Bridge WhatsApp en vivo (`bridge/` + `backend/app/routers/bridge.py`)

Container Node.js con whatsapp-web.js mantiene una sesión persistente en `data/bridge/`. Captura mensajes entrantes y salientes (multi-device sync) en tiempo real, descarga media a MinIO, postea metadata al backend `/api/bridge/whatsapp/ingest`. El backend inserta `core.items` con `source='whatsapp'` y encola los jobs de procesamiento correspondientes (transcribe si es audio, etc.).

5.2 Import histórico de exports `.txt` (`backend/app/services/whatsapp_parser.py`)

Sube un export de chat de WhatsApp (sin media). El parser detecta fechas, autores y mensajes; mapea participantes a personas canónicas y crea/actualiza items. Útil para cargar histórico de los últimos años.

5.3 Uploads manuales (`backend/app/routers/{transcribe,extract,images}.py`)

Páginas de Streamlit para subir audios, documentos (PDF/DOCX/XLSX) e imágenes sueltas. Mismo destino: `core.items` + media en MinIO + jobs encolados.

Personas y conversaciones canónicas:

El sistema mantiene `core.personas` con `nombre_canonico` + `alias` (jsonb) + `telefono`. Cuando llega un mensaje, se resuelve la persona por teléfono → match canónico → si no existe, se crea. El campo `seguir` (default `True` en conversaciones) permite excluir chats irrelevantes del procesamiento masivo.

Importar vCard (`backend/app/services/vcard_parser.py`) ayuda a pre-cargar 9 9 3 contactos con sus nombres reales, así los chats nuevos aparecen ya con nombre humano en lugar de un teléfono.

6. Pipeline de procesamiento

Cinco etapas en cola, drenadas por un **worker continuo** (`backend/app/services/queue_worker.py`) que vive en el lifespan del backend FastAPI. Cada `30s` (configurable) procesa un batch de cada etapa.

6.1 Tabla de etapas

Etapas	Input	Output	Modelo	Batch	Dónde corre
<code>transcribe</code>	Audio en MinIO	<code>item.contenido</code> con texto + <code>transcripcion_at</code>	Whisper Large V3 Turbo (faster_whisper, CPU int 8)	5	CPU multinúcleo
<code>extract</code>	PDF/DOCX/XLSX en MinIO	<code>item.contenido</code> con texto plano	pdfplumber / python-docx / openpyxl	5	CPU
<code>caption</code>	Imagen en MinIO	<code>item.contenido</code> con <code>categoria + OCR + descripcion + entidades</code>	qwen 3 -vl: 8 b	3	GPU
<code>embed</code>	Texto en <code>item.contenido</code>	Vector <code>2560</code> -dim en Qdrant <code>messages + facts</code>	qwen 3 -embedding: 4 b	50	GPU (worker)
<code>tagger</code>	<code>item.contenido</code> (texto)	Resumen, tono, sentimiento, personas/empresas, facts, promesas, transacciones, menciones	qwen 3 : 8 b	3	GPU

6.2 Encadenamiento

Cuando entra un item nuevo:

```
WhatsApp/Upload → core.items → encola → transcribe (si es audio)
                                     → extract (si es doc)
                                     → caption (si es imagen)
                                     → embed (todos los items con texto)

embed completa → encola → tagger (si tagged_at IS NULL)
tagger completa → encola → embed (si creó facts/promesas/transacciones)
                [El re-embed garantiza que los nuevos facts entren a la collection `facts` de Qdrant – sino el retriever no los ve]
```

El re-encadenado `tagger → embed` es lo que hace que el chat empiece a tener resultados de alta calidad: el embedding del fact “Hernan reporta problema con Outlook” matchea con score `~ 0.70` contra la query “qué no le anda a Hernan”, contra `~ 0.60` del mensaje crudo (ver § 8.4).

6.3 Por qué caption corre solo de noche

El `qwen3-vl:8b` pesa `6.14` GB en VRAM. Si se carga durante el día, no queda lugar para que el chat tenga el `qwen3:8b` (`5.23` GB) caliente — Ollama haría swap entre los dos en cada interacción. La ventana `02:00-06:00` garantiza que ningún VLM compita con el chat. Configurable en `.env`: `WORKER_CAPTION_HOUR_START`, `WORKER_CAPTION_HOUR_END`.

Implementación: `queue_worker._caption_en_ventana()` chequea la hora local antes de delegar a `imager.procesar_jobs`. Fuera de ventana, devuelve `{"saltado": "fuera_de_ventana"}`.

6.4 Auto-priorización del tagger

El `tagger.procesar_jobs` ordena por `Item.fecha DESC`, `Job.created_at ASC`. Items más recientes se procesan primero — POC: lo último que llega es lo más útil para validar.

6.5 Throughput medido (al 2026-05-16)

- **Whisper en CPU:** `~5-10` s por minuto de audio. Audios típicos de WhatsApp (`~30-60` s) tardan `~30-60` s en transcribirse. No es interactivo pero está bien para batch.
 - **Tagger con qwen3:8b en GPU:** `~3-5` s por item con `batch=3` → `~6 items/min sostenidos`. Para los `75.700` items totales son `~210` horas (`~9` días). Para los últimos `2` días (`~500` items) son `~80` min — confirmado en sesión real.
 - **Embed batch=50:** `~12` s por batch de `50` mensajes → `~250 items/min`. No es cuello de botella.
 - **Caption nocturno con qwen3-vl en GPU:** `~5-10` s por imagen, `batch=3`, ventana `4` h → `~7.000` imágenes por noche en teoría.
-

7. Almacenamiento

7.1 PostgreSQL (5 schemas)

Definidos en `docker/postgres/init.sql` y poblados con migraciones Alembic en `backend/alembic/versions/`.

Schema	Tablas principales	Para qué
core	personas, empresas, conversaciones, items, facts, promesas, transacciones, menciones	Núcleo del modelo de dominio
media	attachments	Metadata de archivos binarios (los binarios viven en MinIO)
processing	jobs	Cola de tareas para el worker continuo
analytics	(reservado para dinámica conversacional, salud relacional — Sprint 9 +)	
audit	(reservado para logs sensibles)	

Datos actuales: - 7 5 . 7 0 2 items, 9 9 3 personas, 7 9 empresas, 1 1 3 conversaciones - 8 7 1 facts, 2 8 promesas, 1 1 transacciones, 2 9 7 menciones - 1 0 3 attachments en `media.attachments`

7.2 Qdrant (vectores)

Collection	Dim	Distance	Points	Para qué
messages	2 5 6 0	Cosine	7 5 . 6 8 3	Embedding del texto del item (uno por item)
facts	2 5 6 0	Cosine	7 8 0	Embedding de cada hecho extraído por el tagger

El retriever consulta **ambas** y mergea por score. Los facts suelen ganar para queries semánticas porque son texto pre-estructurado (ver § 8).

7.3 MinIO (Vault)

Bucket	Contenido	Estructura
raw	Audios <code>.opus</code> , imágenes, PDFs originales	<code>{source}/{año}/{mes}/{tipo}/{sha256}.{ext}</code>
derived	Transcripciones, thumbnails, OCR results	mismo path con sufijos
exports	Exports manuales (.txt de WhatsApp, etc.)	flat

Identificador por hash SHA- 2 5 6 → **dedup automática** (mismo archivo subido dos veces apunta al mismo objeto).

8. Pipeline de Q&A (chat)

`backend/app/services/chat.py` y `retriever.py`. Una pregunta del usuario pasa por 4 fases:

8.1 Query understanding (pre-procesamiento)

`_analizar_pregunta()` llama a qwen 3 : 8 b con `format=json` pidiendo:

```
{
  "personas": ["nombres propios mencionados"],
  "query_expandida": "reformulación con sinónimos / descripción formal"
}
```

Caso de ejemplo: “qué no le anda a Hernan” → - `personas` : ["Hernan"] - `query_expandida` : “Hernan tiene problemas, fallas o cosas que no funcionan, posiblemente con algún dispositivo, sistema o situación que no le anda bien”

Esto resuelve argentinismos coloquiales que el embedder no conecta bien con descripciones formales. **Costo:** 1 - 2 \$ extra por chat. **Beneficio:** scores 1 0 × mejores en queries cortas.

8.2 Resolución de entidades

`_resolver_personas()` busca cada nombre en `core.personas` por `nombre_canonico ILIKE` o `alias jsonb`. Si hay **exactamente un match**, agrega `persona_id` como filtro al Qdrant (vía `must` clause). Si hay ambigüedad (ej. “Hernan” matchea 6 personas), no filtra — confía en la query expandida.

8.3 Retrieval híbrido

`retriever.recuperar()`:

1. Embed de la `query_expandida` con qwen 3 -embedding: 4 b — **ejecutado en CPU** (`num_gpu=0` vía `force_cpu=True` en `OllamaService.embed`). Por qué CPU: ver § 1 0 .
2. Búsqueda en `messages` (k= 1 2 default) + búsqueda en `facts` (k= 8 default), con filtros opcionales (persona, conversación, rango fechas).
3. Mergeo por score y ordenado descendente.
4. Refresh de metadata desde Postgres (nombre canónico actualizado, nombre del chat).

8.4 Generación con citas

`chat.responder()` arma el contexto con cada fragmento numerado, le pasa a qwen 3 : 8 b con prompt estricto: “Respondé usando ÚNICAMENTE la info de los fragmentos. Citá las fuentes con [n] . Si no alcanza, decilo.”

Métrica real del caso Hernán (audio: “no me está guardando el Oulu los elementos enviados”):

Estado del sistema	Top result	Score	Respuesta del chat
Sin tagger ni query expansion	(no aparece en top 20)	—	“No encuentro esa información”
Solo query expansion	Audio en # 1 (message)	0.56	Correcta pero confusa, mezcla SIAP
Tagger + query expansion	2 facts del audio en # 1 y # 2	0.70 / 0.69	Limpia y precisa, cita [1] [2][3]

8.5 Latencias medidas

Estado	Tiempo total
Cold (primer chat después de restart)	11 - 25 s
Warm (qwen 3 : 8 b ya cargado)	2.5 - 5 s

9. Métricas y volumen

9.1 Datos en el sistema

```
items_totales: 75.702
items_embebidos: 75.690 (99,98%)
items_taggeados: 617 (0,8%) ← el tagger es nuevo, está corriendo
```

9.2 Distribución por tipo de media

```
texto: 73.354
imagen: 1.695 ← procesados de noche con qwen3-vl
audio: 435 ← procesados por Whisper CPU
video: 166
contacto: 31
documento: 11
sticker: 10
```

9.3 Salidas estructuradas del tagger (muestra real)

Promesas detectadas con confianza ≥ 0.9 : - *"coordinar para que vaya a abrir el lugar de la caja fuerte"* — Julián García Urbana, plazo: lunes - *"llevar una impresora de reemplazo"* — Mariano Di Nucci, plazo: lunes/martes - *"firmar la oferta por el departamento"* — Fabian firpo, plazo: semana que viene - *"entregar el trabajo a Secre"* — Eugenia Clínica, plazo: miércoles

Transacciones detectadas: - \$ 4 2.0 0 0 ARS, *"Web Hosting Plan 3"*, egreso, confianza 0.9 - \$ 3 2.3 0 0 ARS, transferencia, ingreso, confianza 0.9 - \$ 5.7 8 3 ARS, *"cuota VISA"*, egreso, confianza 0.9

Resúmenes de imágenes (qwen 3 -vl): - *"Captura de pantalla de una transferencia bancaria enviada por Pablo Luciano Montegriffo a Damian Federico Orozco por \$ 3 2.3 0 0"* - *"Pantalla de una compra en donweb.com, mostrando un plan de Web Hosting"*

9.4 Estado de colas al momento del snapshot

```
caption: pendiente=2 en_proceso=1 fallido=1 completado=38
tagger: pendiente=631 en_proceso=2 completado=538
embed: completado=1.029
transcribe: completado=57 en_proceso=1
```

10. Decisiones técnicas con tradeoffs

10.1 Whisper en CPU (decisión clave, 2026-05-15)

Problema: `:latest-gpu` con `large-v3-turbo` ocupaba 4.3 GB de VRAM permanente, aun sin transcribir nada. Eso dejaba 3.3 GB libres para Ollama, insuficiente para tener qwen 3 : 8 b + qwen 3 -embedding: 4 b cargados juntos. El chat hacía **swap completo entre modelos en cada mensaje** (~ 10 - 15 s por swap).

Decisión: Cambiar a imagen CPU (`:latest` sin `-gpu`), sin GPU access en compose.

Trade-off: - ☒ Libera 4.3 GB de VRAM permanente - ☒ qwen 3 : 8 b ahora cabe 100 % en GPU - ☒ Transcripción ~ 5 - 10 s por minuto de audio (vs ~ 0.1 s/min en GPU) - ☒ No es un problema porque la transcripción es background, no interactiva

Resultado: chat warm pasó de 25 s → 2.5 s por mensaje.

10.2 Embedding del chat en CPU

Problema: incluso con Whisper liberado, qwen 3 : 8 b (5.23 GB) + qwen 3 -embedding: 4 b (3.86 GB en VRAM) = 9.1 GB > 8 GB. No caben juntos. Cada chat hace embed → unload chat → load embed → ... → swap completo.

Decisión: en `retriever.recuperar()`, embed de la query con `force_cpu=True` (pasa `num_gpu=0` a Ollama). El qwen 3 : 8 b queda caliente en GPU permanentemente. El worker continuo sigue embebiendo batches en GPU porque corre en background.

Trade-off: - ☒ Embedding de la query tarda ~ 1 - 2 s en CPU (vs ms en GPU) - ☒ Sin swap → chat consistente - ☒ Worker no afectado

10.3 Caption en ventana nocturna 02:00-06:00

Problema: `qwen3-vl:8b` ocupa 6 GB en VRAM. Si se carga durante el día, expulsa al qwen 3 : 8 b y rompe el chat.

Decisión: configurable `worker_caption_hour_start/end` en `config.py`. Por defecto 02:00 - 06:00 hora local Argentina. Fuera de la ventana, la etapa `caption` se saltea (los jobs quedan pendientes hasta la siguiente ventana).

Trade-off: - ☒ Chat libre durante el día - ☒ Imágenes nuevas no se procesan hasta la madrugada (latencia hasta 24 h) - ☒ Suficiente para POC; la mayoría del valor está en texto y audios

10.4 Tagger en el worker (vs ejecución manual)

Problema histórico: hasta 2026-05-15, el servicio `tagger.taggear_item` existía pero **no estaba integrado al worker**. Resultado: 61.887 items embebidos en Qdrant, 0 taggeados. El chat dependía únicamente de embedding crudo, que es flojo con argentinismos.

Decisión: agregar etapa `tagger` al `queue_worker`. Hook automático: cuando un embed se completa, encola un tagger job. Cuando un tagger termina exitoso (creó facts/promesas/transacciones), encola un re-embed para que los nuevos artefactos entren a Qdrant.

Trade-off: - ☒ Pipeline coherente con el diseño original (Sprint 3) - ☒ El chat empieza a tener resultados de calidad alta - ☒ Tagger compete por qwen 3 : 8 b con el chat; mientras corre, los chats van más lentos - ☒ Aceptado para POC: priorizamos calidad del dato sobre fluidez del chat

10.5 Query expansion en el chat (vs cambiar el embedding model)

Problema: `qwen3-embedding:4b` no conecta bien argentinismos ("no le anda" ≠ "no funciona" en su espacio semántico).

Decisión: agregar 1 llamada extra a qwen 3 : 8 b por chat para expandir la query. Más simple que cambiar el modelo de embedding (que requeriría re-embeber los 7 5 K items).

Trade-off: - ☒ + 1 - 2 s por chat - ☒ Mejora masiva del ranking sin re-embeber nada - ☒ Trivial de revertir si en el futuro se cambia el embedding model

10.6 Modelos múltiples descargados pero solo 3 en uso

Hay 9 modelos descargados (gemma 3 : 1 2 b, gemma 4 :e 4 b, aya-expanse: 8 b, etc.). Solo se usan 3 productivamente (qwen 3 : 8 b, qwen 3 -vl: 8 b, qwen 3 -embedding: 4 b). Los otros son **opciones para A/B test** si quisiéramos cambiar — descargados durante el benchmark de Sprint 0 .

11. Limitaciones conocidas y deuda técnica

Limitación	Impacto	Mitigación posible
El chat-during-day se hace lento mientras corre el backfill del tagger	UX pobre durante setup inicial	Mover tagger a ventana nocturna también (~mismo patrón que caption)
Solo 0,8 % de items taggeados al momento	Chat solo aprovecha facts en items recientes	Backfill nocturno corriendo, ~ 9 días para todo el histórico
Sin facts para imágenes hasta procesarlas de noche	Búsquedas sobre fotos no encuentran nada hasta el primer ciclo de caption	Esperar el primer batch nocturno
Whisper en CPU es ~ 50x más lento que en GPU	Audios nuevos tardan 30 - 60 s en estar disponibles para chat	Aceptable (la mayoría se procesa en background)
Query expansion suma 1 - 2 s por chat	UX	Caché de queries comunes (no implementado)
Tagger usa el LLM "thinking" model de qwen 3 con <code>think=False</code>	Si el modelo lo ignora, los outputs salen con cadena <code><think></code> rara	Manejo robusto en <code>_extraer_json</code> que strippea esas etiquetas
Frontend Streamlit es para validación, no producción	Lento, recarga full-page, no es bonito	Panel de escritorio en <code>panel/</code> cubre control. Para usuarios finales, panel propio (Reflex o similar) en Sprint posterior
Single-node, sin réplica, sin backups automáticos	Si se rompe el disco, se pierde todo	El usuario backupea manualmente. Sprint posterior podría agregar S3 /duplicidad

12. Opciones a evaluar con el equipo

A. Hardware: ¿upgrade de GPU?

Pasar a una GPU con 16 GB+ VRAM (4080, A5000, etc.) cambia las restricciones de \$2: - Whisper podría volver a GPU - qwen 3: 8b + qwen 3-embedding: 4b + qwen 3-vl: 8b cargados simultáneamente - Eliminaría toda la gymnastics de CPU offload y ventana nocturna - Tradeoff: costo (\$500-2000 USD)

B. Cambio de embedding model

Pasar de `qwen3-embedding:4b` (2.5 GB, 2560 dim) a uno chico tipo `bge-m3` (568 MB, 1024 dim) o `omic-embed-text` (270 MB, 768 dim): - ☒ Caben con qwen 3: 8b sin tocar - ☒ Quality benchmark a favor de bge-m3 en multilingüe (incluye español) - ☒ Requiere re-embeber 75K items (~4-6h) - ☒ Recrear collections en Qdrant con nuevo dim

C. Migrar a otro stack de LLM serving

- **vLLM** con un modelo en `/tmp` instead of Ollama
- Pro: mejor throughput batch, server-side batching real
- Contra: menos plug-and-play, harder Windows/WSL 2 setup

D. RAG arquitectura híbrida

Hoy es full-semantic. Agregar **BM 25** + **full-text search** de Postgres como complemento: - Para queries con nombres propios, números, fechas → BM 25 es mejor - Para queries conceptuales → semantic es mejor - Mergear ambos con reciprocal rank fusion (RRF)

E. Tagger paralelo

Ollama soporta `OLLAMA_NUM_PARALLEL > 1`. Si el modelo cabe holgado (qwen 3: 4b en lugar de qwen 3: 8b), podríamos paralelizar el tagger: - ☒ ~ 2-3x throughput - ☒ Calidad menor con 4B - ☒ Pierde determinismo

F. Quality / determinismo del tagger

Hoy `temperature=0.1`. Trade-off entre creativity (captar matices) y reproducibilidad. Para POC, bajar a 0.0 podría dar resultados más auditables.

G. Ampliar fuentes de datos

El diseño está pensado para más fuentes. En orden de complejidad: - **Gmail** (IMAP + parser MIME) — siguiente más natural, similar a WhatsApp - **Calendar** — agregaría contexto temporal a items - **Telegram** — similar a WhatsApp con MTPROTO - **Drive** — más complejo, pero los docs son fáciles de procesar con `extract`

13. Glosario rápido

- **Item** — Unidad mínima de información en el sistema (un mensaje, un email, una nota). Tabla `core.items`.
 - **Persona canónica** — Una persona del mundo real, con N alias/teléfonos resueltos a la misma fila. Tabla `core.personas`.
 - **Fact** — Hecho atómico extraído por el tagger del contenido de un item. Tabla `core.facts`. Va a Qdrant `facts`.
 - **Promesa** — Compromiso explícito detectado por el tagger ("te lo paso el lunes"). Tabla `core.promesas`.
 - **nivel_procesamiento** — `0` = ingestado pero sin tagging; `1` = taggeado.
 - **tagged_at**, **embedded_at**, **transcripcion_at** — Marcas en `item.datos` (jsonb) que indican qué etapas se completaron. Permite saltarlas en re-runs.
 - **Ventana caption** — Rango horario diario en que la etapa `caption` puede usar GPU. Default `0 2 - 0 6` local.
 - **force_cpu** — Flag en `OllamaService.embed` que mete `num_gpu=0` en la request, forzando embedding en CPU.
-

14. Referencias al código

Mapa rápido para navegación:

```

backend/
├── app/
│   ├── config.py                # toda la config via pydantic-settings
│   ├── main.py                  # FastAPI app + lifespan (worker startup)
│   ├── core/logging.py
│   ├── db/session.py
│   ├── models/
│   │   ├── core.py              # Persona, Empresa, Conversacion, Item
│   │   ├── tagging.py           # Fact, Promesa, Transaccion, Mencion
│   │   ├── media.py             # Attachment
│   │   └── processing.py        # Job
│   ├── routers/
│   │   ├── bridge.py            # /api/bridge/* - ingesta de WhatsApp
│   │   ├── tagger.py            # /api/tagger/* - endpoints manuales
│   │   ├── chat.py              # /api/chat - Q&A
│   │   ├── embeddings.py        # /api/embeddings/*
│   │   ├── transcribe.py        # /api/transcribe/*
│   │   ├── extract.py           # /api/extract/*
│   │   ├── images.py            # /api/images/*
│   │   ├── worker.py            # /api/worker/{status,pause,resume,tick}
│   │   ├── panel.py             # /api/panel/* - endpoints para panel
│   │   ├── imports.py           # /api/import/whatsapp/*
│   │   ├── contacts.py, conversations.py
│   │   └── health.py
│   └── services/
│       ├── queue_worker.py       # ★ worker continuo (todas las etapas)
│       ├── transcriber.py        # etapa transcribe
│       ├── extractor.py          # etapa extract
│       ├── imager.py            # etapa caption
│       ├── embedder.py          # etapa embed
│       ├── tagger.py             # ★ etapa tagger (incl. runtime_config)
│       ├── chat.py              # ★ pipeline de Q&A
│       ├── retriever.py          # búsqueda híbrida en Qdrant
│       ├── ollama_client.py      # wrapper con force_cpu
│       ├── qdrant_client.py
│       ├── minio_client.py       # VaultStorage
│       ├── whisper_client.py
│       ├── whatsapp_parser.py
│       ├── vcard_parser.py
│       └── phones.py
├── alembic/versions/            # 4 migraciones (Sprints 1, 2.5, 3)

frontend/                        # Streamlit (validación + UIs)
├── pages/{1..13}/*.py
└── lib/api_client.py

bridge/                          # Node.js whatsapp-web.js (Sprint 2)

panel/                           # ★ App de escritorio (PySide6)
├── secondbrain_panel/
│   ├── api_client.py            # cliente HTTP al backend
│   ├── docker_client.py         # wrapper docker compose
│   ├── system_stats.py          # CPU/GPU/RAM via psutil + nvidia-smi
│   ├── main_window.py           # QMainWindow con 7 tabs
│   └── tabs/
│       ├── sistema.py           # CPU/RAM/GPU/Ollama
│       ├── servicios.py         # containers Docker
│       ├── worker.py            # status del worker
│       ├── colas.py             # contadores processing.jobs
│       ├── chats.py             # acciones por conversación
│       ├── tagger.py            # trigger manual
│       └── configuracion.py      # batches, modelo del tagger, ventana caption

docs/
├── sprints.md                   # plan + 18 queries de referencia
├── architecture.md              # arquitectura conceptual original
├── setup-windows.md             # cómo levantar en Windows
└── pipeline.md                  # ← este documento

scripts/
└── benchmark_tagger.py          # comparador A/B de modelos LLM

```